คู่มือโครงสร้างฐานข้อมูลและข้อกำหนดทางเทคนิค

โปรเจกต์: ระบบบริหารจัดการร้านเกมคาเฟ่ (LETSPLAY GAMING) • Laravel 12 & SQLite

Database Architecture Guide

1. ภาพรวมโครงสร้างฐานข้อมูล (Database Overview)

ฐานข้อมูลจัดเก็บในไฟล์ `database/database.sqlite` ประกอบด้วย **11 ตารางหลัก** ในการดำเนินงานของร้านเกมคาเฟ่ แบ่งออกเป็น 4 กลุ่มฟังก์ชันหลัก: (1) ผู้ใช้และกระเป๋าเงิน (User & Wallet), (2) โซนและเครื่องคอมพิวเตอร์ (Zones, Seats & Sessions), (3) แพ็กเกจชั่วโมง (Time Packages), และ (4) สั่งอาหารและสต็อกสินค้า (Food, Orders & Products)

2. รายละเอียด 11 ตารางหลัก (Tables Specification)

1. ตาราง users

`App\Models\User`

เก็บข้อมูลบัญชีผู้ใช้งานระบบทั้งหมด ทั้งลูกค้า (Customer), พนักงานร้าน (Staff) และผู้จัดการร้าน (Admin)

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสผู้ใช้งาน
name	`varchar(255)`	Required	ชื่อ-นามสกุล หรือชื่อเรียก
username	`varchar(50)`	Unique	ชื่อบัญชีสำหรับเข้าสู่ระบบ
email	`varchar(255)`	Nullable, Unique	อีเมล
phone	`varchar(20)`	Nullable	เบอร์โทรศัพท์
role	`varchar(20)`	Default: 'customer'	สิทธิ์การใช้งาน (customer, staff, admin)
balance	`decimal(10,2)`	Default: 0.00	ยอดเงินคงเหลือในกระเป๋าเงินดิจิทัล (Wallet)
password	`varchar(255)`	Hashed	รหัสผ่านเข้าสู่ระบบ

ความสัมพันธ์ (Eloquent Relationships): hasMany(WalletTransaction), hasMany(UserPackage), belongsToMany(Package, 'user_packages'), hasMany(SeatSession), hasOne(SeatSession active), hasMany(Order)

2. ตาราง wallet_transactions

`App\Models\WalletTransaction`

ทำหน้าที่เป็น Audit Log บันทึกประวัติการเปลี่ยนแปลงยอดเงินในกระเป๋าเงินทุกรายการ เพื่อความโปร่งใสและตรวจสอบย้อนหลังได้

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสรายการธุรกรรม
user_id	`bigint`	FK -> users.id (Restrict)	ผู้ใช้งานเจ้าของรายการ
type	`varchar(20)`	topup / deduct / refund	ประเภทธุรกรรม (เติมเงิน, หักเงิน, คืนเงิน)
amount	`decimal(10,2)`	Required	จำนวนเงินของรายการ
ref_type	`varchar(50)`	Nullable	ที่มาของรายการ (session, order, package_purchase, cash_topup, qr_topup)
ref_id	`bigint`	Nullable	ไอดีอ้างอิงจากตารางต้นทาง เช่น session_id หรือ order_id

ความสัมพันธ์: belongsTo(User)

3. ตาราง zones

`App\Models\Zone`

เก็บข้อมูลโซนเครื่องคอมพิวเตอร์และอัตราค่าบริการต่อชั่วโมงแยกตามประเภทห้องและสเปกเครื่อง

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสโซน
name	`varchar(100)`	Required	ชื่อโซน (เช่น Standard Zone 165Hz, VIP 240Hz, Duo Private Room)
hourly_rate	`decimal(10,2)`	Required	อัตราค่าบริการต่อชั่วโมง (40.00, 60.00, 100.00 บาท)
description	`text`	Nullable	คำอธิบายสเปกเครื่องหรือบรรยากาศ

ความสัมพันธ์: hasMany(Seat), hasMany(Package)

4. ตาราง seats

`App\Models\Seat`

เก็บข้อมูลตำแหน่งเครื่องคอมพิวเตอร์ในร้านและสถานะการใช้งานปัจจุบัน

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสที่นั่ง/เครื่อง
zone_id	`bigint`	FK -> zones.id (Restrict)	โซนที่เครื่องนี้ตั้งอยู่
seat_number	`varchar(20)`	Unique	หมายเลขเครื่อง เช่น STD-01, VIP-02, DUO-01
status	`varchar(20)`	Default: 'available'	สถานะ (available ว่าง, occupied มีคนเล่น, maintenance ซ่อมบำรุง)

ความสัมพันธ์: belongsTo(Zone), hasMany(SeatSession), hasOne(SeatSession active), hasMany(Order)

5. ตาราง seat_sessions

`App\Models\SeatSession`

บันทึกรอบการเปิดใช้งานเครื่องคอมพิวเตอร์ ตั้งแต่เริ่ม Check-in จนถึง Check-out ปิดเครื่อง

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสเซสชันการใช้งาน
user_id	`bigint`	FK -> users.id (Restrict)	ลูกค้าผู้เปิดใช้งานเครื่อง
seat_id	`bigint`	FK -> seats.id (Restrict)	เครื่องคอมพิวเตอร์ที่เปิดใช้งาน
user_package_id	`bigint`	Nullable, FK -> user_packages.id	แพ็กเกจที่ใช้หักเวลา (หากเป็น null คือเล่นแบบ Pay As You Go)
rate_snapshot	`decimal(10,2)`	Required	ราคาต่อชั่วโมง ณ วินาทีที่เปิดเครื่อง (Snapshot ป้องกันการปรับราคาภายหลัง)
start_time	`datetime`	Required	เวลาเริ่มเปิดเครื่อง
end_time	`datetime`	Nullable	เวลาสิ้นสุดการใช้งาน (เช็คเอาท์)
total_cost	`decimal(10,2)`	Default: 0.00	ยอดเงินค่าเครื่องที่คิดเงินจริงเมื่อเช็คเอาท์
status	`varchar(20)`	Default: 'active'	สถานะเซสชัน (active กำลังเล่น, completed สิ้นสุดแล้ว)

ความสัมพันธ์: belongsTo(User), belongsTo(Seat), belongsTo(UserPackage), hasMany(Order)

6. ตาราง packages

`App\Models\Package`

เก็บข้อมูลรูปแบบแพ็กเกจชั่วโมงที่ร้านเปิดขายให้ลูกค้าซื้อล่วงหน้าในราคาโปรโมชัน

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสแพ็กเกจ
zone_id	`bigint`	Nullable, FK -> zones.id	โซนที่แพ็กเกจนี้ใช้งานได้ (null คือใช้ได้ทุกโซน)
name	`varchar(100)`	Required	ชื่อแพ็กเกจ เช่น Standard 3 ชม., VIP 5 ชม.
duration_hours	`integer`	Required	จำนวนชั่วโมงการเล่นที่ได้รับ
price	`decimal(10,2)`	Required	ราคาขายของแพ็กเกจ

ความสัมพันธ์: belongsTo(Zone), hasMany(UserPackage)

7. ตาราง user_packages

`App\Models\UserPackage`

เก็บข้อมูลแพ็กเกจที่ลูกค้าซื้อเก็บไว้ในบัญชี และจำนวนนาทีคงเหลือที่ยังเล่นได้ (Rich Pivot Table)

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสแพ็กเกจของผู้ใช้
user_id	`bigint`	FK -> users.id (Restrict)	ลูกค้าเจ้าของแพ็กเกจ
package_id	`bigint`	FK -> packages.id (Restrict)	แพ็กเกจที่ซื้อ
remaining_minutes	`integer`	Required	จำนวนนาทีคงเหลือที่ยังเล่นได้
purchased_at	`datetime`	Default: NOW	วันเวลาที่ซื้อแพ็กเกจ
expired_at	`datetime`	Nullable	วันเวลาหมดอายุของแพ็กเกจ

ความสัมพันธ์: belongsTo(User), belongsTo(Package), hasMany(SeatSession)

8. ตาราง categories

`App\Models\Category`

หมวดหมู่อาหารและเครื่องดื่มสำหรับแยกประเภทในหน้าเมนูอาหาร

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสหมวดหมู่
name	`varchar(100)`	Required	ชื่อหมวดหมู่ เช่น อาหารจานเดียว, ของทานเล่น, เครื่องดื่ม

ความสัมพันธ์: hasMany(Product)

9. ตาราง products

`App\Models\Product`

เก็บรายการอาหาร ขนม เครื่องดื่ม และจำนวนสต็อกสินค้าคงเหลือในร้าน

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสสินค้า
category_id	`bigint`	FK -> categories.id (Restrict)	หมวดหมู่สินค้า
name	`varchar(150)`	Required	ชื่อสินค้า
description	`text`	Nullable	รายละเอียดสินค้า
price	`decimal(10,2)`	Required	ราคาขายต่อหน่วย
stock_quantity	`integer`	Default: 0	จำนวนคงเหลือในสต็อก (ตัดสต็อกอัตโนมัติเมื่อสั่งซื้อ)
image	`varchar(255)`	Nullable	พาธไฟล์รูปภาพ

ความสัมพันธ์: belongsTo(Category), hasMany(OrderItem)

10. ตาราง orders

`App\Models\Order`

เก็บข้อมูลคำสั่งซื้ออาหารและเครื่องดื่มที่ลูกค้ากดสั่งจากหน้าเครื่องคอมพิวเตอร์ส่งเข้าครัว

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสออเดอร์
user_id	`bigint`	FK -> users.id (Restrict)	ลูกค้าผู้สั่งอาหาร
seat_id	`bigint`	FK -> seats.id (Restrict)	หมายเลขเครื่องที่ให้ไปส่งอาหาร
session_id	`bigint`	Nullable, FK -> seat_sessions.id	รอบเปิดเครื่องขณะสั่ง
total_amount	`decimal(10,2)`	Default: 0.00	ยอดรวมค่าอาหารทั้งหมด
payment_method	`varchar(20)`	wallet / promptpay / cash	ช่องทางชำระเงิน
payment_status	`varchar(20)`	pending / paid / cancelled	สถานะการชำระเงิน
order_status	`varchar(20)`	pending / preparing / served / cancelled	สถานะคิวครัวพนักงาน

ความสัมพันธ์: belongsTo(User), belongsTo(Seat), belongsTo(SeatSession), hasMany(OrderItem)

11. ตาราง order_items

`App\Models\OrderItem`

เก็บรายการย่อยของสินค้าแต่ละชิ้นในออเดอร์ (Line Items)

คอลัมน์	ชนิดข้อมูล	คุณสมบัติ	คำอธิบาย
id	`bigint`	PK, Auto Increment	รหัสรายการย่อย
order_id	`bigint`	FK -> orders.id (Cascade)	เชื่อมกับออเดอร์หลัก (ลบออเดอร์แล้วรายการย่อยถูกลบตาม)
product_id	`bigint`	FK -> products.id (Restrict)	สินค้าที่สั่งซื้อ
quantity	`integer`	Required	จำนวนชิ้นที่สั่ง
unit_price	`decimal(10,2)`	Required	ราคาต่อหน่วย ณ เวลาสั่งซื้อ (Snapshot ป้องกันการปรับราคาในอนาคต)
subtotal	`decimal(10,2)`	Required	ราคารวมรายการย่อย (quantity × unit_price)

ความสัมพันธ์: belongsTo(Order), belongsTo(Product)

3. ข้อกำหนดทางเทคนิคและหลักการออกแบบที่ควรรู้ (Technical Architecture)

1. นโยบาย Foreign Key Constraints

• **restrictOnDelete():** ใช้กับความสัมพันธ์เกือบทั้งหมดเพื่อรักษาความสมบูรณ์ของข้อมูลทางการเงิน (Data Integrity) ป้องกันการลบ User หรือ Seat ที่มีประวัติเงิน/ประวัติการเล่นค้างอยู่

• **cascadeOnDelete():** ใช้เฉพาะ `orders -> order_items` เพราะรายการย่อยเป็นส่วนหนึ่งของหัวบิล เมื่อลบบิล รายการย่อยต้องถูกลบตาม

2. เทคนิค Snapshot Pattern

• **rate_snapshot ใน seat_sessions:** บันทึกราคาต่อชั่วโมงตอนเปิดเครื่อง ป้องกันไม่ให้การขึ้นราคาโซนในอนาคตกระทบเซสชันที่เล่นอยู่

• **unit_price ใน order_items:** บันทึกราคาสินค้า ณ วันที่สั่งซื้อ ป้องกันไม่ให้ยอดบิลในอดีตเปลี่ยนแปลงเมื่อร้านปรับราคาสินค้าในอนาคต

3. โครงสร้าง Rich Pivot Table (user_packages)

ตาราง `user_packages` ไม่ใช่แค่ pivot table จับคู่ไอดี แต่มี Primary Key ของตัวเอง และเก็บ `remaining_minutes`, `purchased_at`, `expired_at` ทำให้ลูกค้า 1 คน สามารถซื้อแพ็กเกจเดิมซ้ำได้หลายครั้งโดยเวลาไม่ปนกัน

4. คุณสมบัติ ACID & DB::transaction()

การทำรายการสำคัญ เช่น สั่งอาหาร มีการใช้ `DB::transaction()` เพื่อหักเงินใน wallet, ตัดสต็อกสินค้า, สร้างออเดอร์พร้อมกัน หากจุดใดล้มเหลว ระบบจะ Rollback คืนค่าทั้งหมด ป้องกันเงินถูกหักแต่ไม่ได้อาหาร

5. กฎธุรกิจและตัวเลขที่ควรจำ (Business Rules)

• **เติมเงิน (Wallet):** ขั้นต่ำ 20 บาท สูงสุด 10,000 บาท

• **Pay As You Go:** ต้องมีเงินในกระเป๋าอย่างน้อย 20.00 บาท จึงจะเปิดเครื่องได้

• **อัตราโซน:** Standard (40 บ./ชม.), VIP (60 บ./ชม.), Duo (100 บ./ชม.)

6. บัญชีทดสอบในระบบ (Seeder Users)

• **admin:** รหัสผ่าน `password` (ยอดเงิน 1,000 บ.)

• **staff:** รหัสผ่าน `password` (ยอดเงิน 500 บ.)

• **customer1:** รหัสผ่าน 1 (ยอดเงิน 20 บ.)

• **customer2:** รหัสผ่าน `password`

ตารางระบบภายในของ Laravel (Laravel System Tables):

• `migrations`: บันทึกประวัติไฟล์ migration ที่รันแล้ว • `sessions`: เก็บข้อมูลเซสชันการล็อกอินในเว็บ • `password_reset_tokens`: เก็บโทเค็นรีเซ็ตรหัสผ่าน • `cache` / `cache_locks`: เก็บข้อมูลแคช • `jobs` / `failed_jobs`: เก็บคิวประมวลผลงานเบื้องหลัง